SIMATS ENGINEERING
ASSIGNMENT – CO2

COURSE NAME: DATABASE MANAGEMENT SYSTEMS
COURSE CODE: CSA05
NAME / REG NO: MONISH.L [1972260035]
NATURE: Individual | Duration: 1 Week | Total Marks: 50

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.

Assignment title: Relational Design and Validation for Courier Handover Completion (SwiftSpan Parcels)

Scenario in brief

Handover sheets mix sender addresses, service labels and repeated coordinator details. One observation is identified by (SenderID, ServiceID, DispatchWave). SenderID determines SenderName and RouteGroup; ServiceID and the unique ServiceCode identify one catalog object and its ServiceName and VolumeWeight; the assignment scope {SenderID, ServiceID} determines CoordinatorID; CoordinatorID determines CoordinatorName and Hub; the full observation key determines the integer Completion (0–100); under the fixed fictional policy Completion determines ServiceBand and ServiceBand determines RouteFactor. The test sizes are 10,000, 100,000 and 600,000 observations (500, 5,000 and 30,000 actors, each with five required catalog items observed in four periods). **Fourteen attributes:** SenderID, SenderName, RouteGroup, ServiceID, ServiceCode, ServiceName, VolumeWeight, DispatchWave, CoordinatorID, CoordinatorName, Hub, Completion, ServiceBand, RouteFactor.

Assumptions I added (the case leaves them open)

- **Required catalog items:** the 40-item catalog is split into 8 route groups G0–G7 of five items each; a sender's RouteGroup fixes its five required services. This is stored in a small reference table `Req(RouteGroup, ServiceID)` that is identical in all three designs and is the 'declared group' used by Query 2.
- **Policy:** Completion 0–44 → band D (RouteFactor 130), 45–64 → C (115), 65–84 → B (100), 85–100 → A (90); RouteFactor is stored as hundredths. VolumeWeight is an integer 1–5. **Threshold** 39 means 'sub-threshold' is Completion < 39.
- **Weighted adjusted index** (Query 1) = Σ(Completion × RouteFactor/100 × VolumeWeight) ÷ ΣVolumeWeight over the selected observations.
- **Data:** Completion ≈ Normal(50 + (ServiceID×7 mod 31), 14), clipped to 0–100; coordinators are assigned as 1 + ((SenderID×7 + ServiceID) mod number-of-coordinators); everything is produced by `random.Random(4211)`. No live organizational data are used.

Execution note (read before the results)

All SQL in Tasks 4, 8, 9 and 12 was **actually executed** on **SQLite 3.45.1** (Python `sqlite3`, foreign keys ON, total run time 57 s) at all three sizes. **A MySQL server could not be installed in the work environment** (package downloads are blocked), so no measurement in this report comes from MySQL. The submission DDL (Appendix B) is written in MySQL 8.0 syntax and was **not executed**; the executed schema differs only in dialect. Timings are SQLite medians of five runs on one shared 2-core container, bytes are SQLite pages (4,096 B) from `dbstat`, and integrity messages are SQLite's. MySQL privileges (`GRANT`/`REVOKE`) are shown as examples only. The FD, closure, key, minimal-cover, chase and normal-form results were **computed by program** (`fd_theory.py`), not only by hand. The two runners are listed in Appendix A.

Signature of the student:



TASK 1 – Design Alternative Relational Schemas (5 Marks)

Design three genuinely different decompositions for SwiftSpan Parcels (flat baseline with a declared logical key, a partial design that still stores operator and policy consequences, and a BCNF design), declare every key and reference, draw them with determinant-based links and justify testing each.

Answer:

1.1 Design D1 – flat baseline

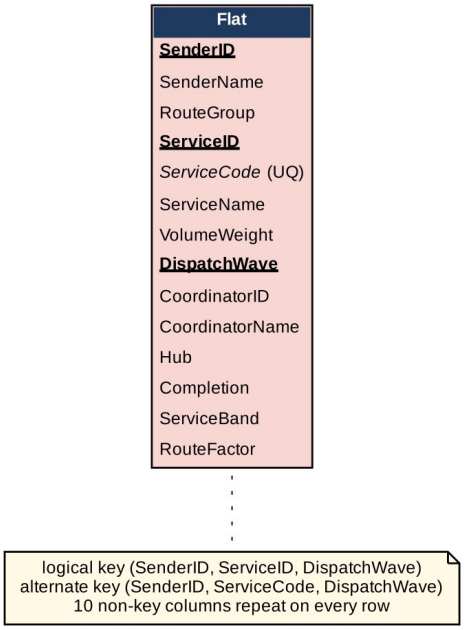


Figure 1 – D1: one 14-column relation; every dependent fact is repeated on every observation

`Flat(SenderID, SenderName, RouteGroup, ServiceID, ServiceCode, ServiceName, VolumeWeight, DispatchWave, CoordinatorID, CoordinatorName, Hub, Completion, ServiceBand, RouteFactor)`. **Declared logical key:** PRIMARY KEY (SenderID, ServiceID, DispatchWave); **alternate key:** UNIQUE (SenderID, ServiceCode, DispatchWave), because ServiceCode ↔ ServiceID. There are no foreign keys: all facts live in one place, so every sender/catalog/operator/policy fact is copied once per observation. Why test it: it needs no joins, so it is the fastest candidate for look-ups; it shows what the stated workload would cost without decomposition.

1.2 Design D2 – partial decomposition

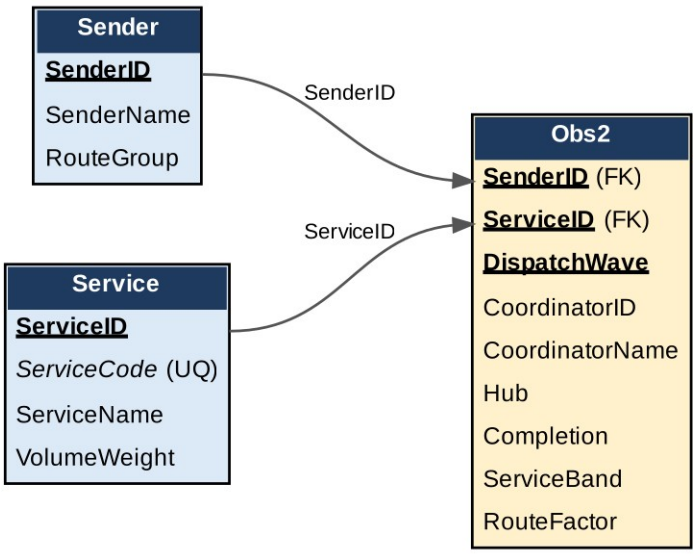


Figure 2 – D2: identity and catalog separated; operator and policy consequences still stored in Obs2

`Sender(SenderID PK, SenderName, RouteGroup)`, `Service(ServiceID PK, ServiceCode UNIQUE, ServiceName, VolumeWeight)`, `Obs2(SenderID FK, ServiceID FK, DispatchWave, CoordinatorID, CoordinatorName, Hub, Completion, ServiceBand, RouteFactor; PK (SenderID, ServiceID, DispatchWave))`. Sender identity and catalog facts are removed from the observation rows, but **CoordinatorName, Hub (via CoordinatorID) and ServiceBand/RouteFactor (via Completion) are still stored redundantly** and there is no separate assignment or policy table. Why test it: it is the common 'half-normalised' design; it needs only two joins, so it shows whether removing identity/catalog repetition is enough.

1.3 Design D3 – BCNF decomposition

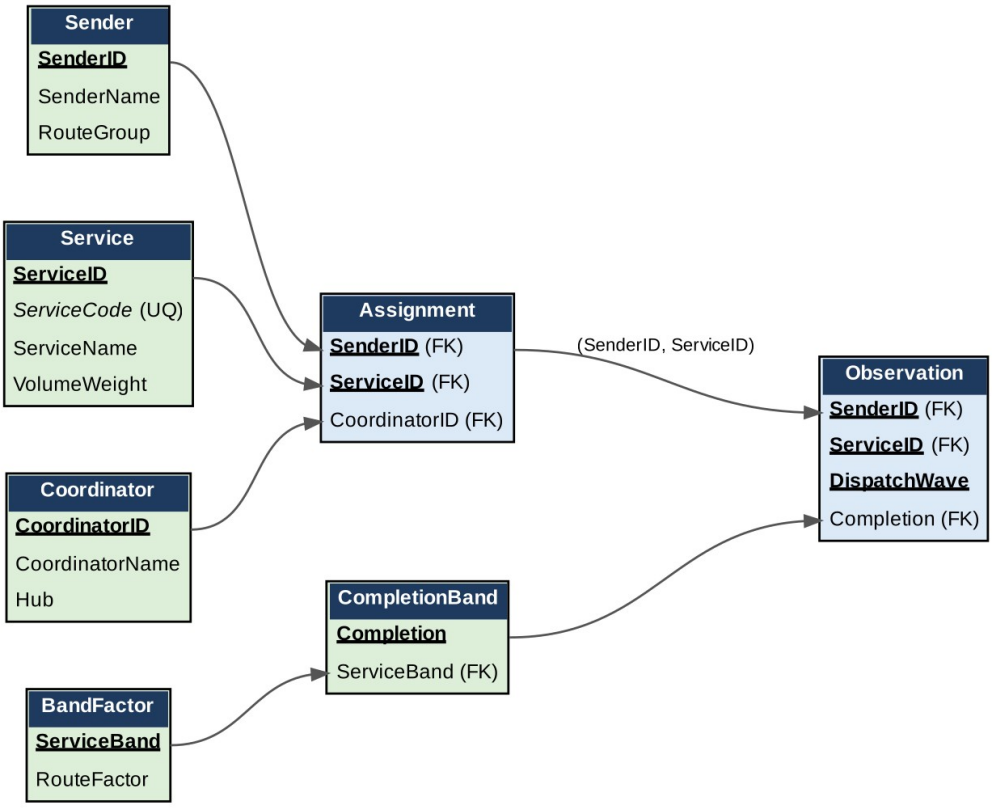


Figure 3 – D3: seven BCNF relations; each arrow is a determinant-to-reference link used in the executed DDL

Relation	Primary key	Other keys	References (all ON DELETE RESTRICT, ON UPDATE CASCADE)
Sender(SenderID, SenderName, RouteGroup)	SenderID	–	–
Service(ServiceID, ServiceCode, ServiceName, VolumeWeight)	ServiceID	UNIQUE ServiceCode	–
Coordinator(CoordinatorID, CoordinatorName, Hub)	CoordinatorID	–	–
Assignment(SenderID, ServiceID, CoordinatorID)	(SenderID, ServiceID)	–	SenderID → Sender; ServiceID → Service; CoordinatorID → Coordinator
Observation(SenderID, ServiceID, DispatchWave, Completion)	(SenderID, ServiceID, DispatchWave)	–	(SenderID, ServiceID) → Assignment; Completion → CompletionBand
CompletionBand(Completion, ServiceBand)	Completion	–	ServiceBand → BandFactor
BandFactor(ServiceBand, RouteFactor)	ServiceBand	–	–

Every non-key fact is stored once, and every determinant of the dependencies is the key of the relation that stores its consequence. The alternate business identifier ServiceCode is a UNIQUE key of Service; Observation reaches the catalog through ServiceID, so the code is never copied. The fixed policy is stored as data (CompletionBand, BandFactor), so a policy change updates a table instead of every observation. Why test it: it is the design the dependencies call for, and the workload shows the price of its joins (six joins to rebuild the 14-column row).

1.4 Why the three are genuinely different

The designs differ in **where facts are placed**, not in names: D1 stores all ten dependent attributes per observation; D2 moves two determinants (SenderID, ServiceID) out but keeps CoordinatorID and Completion as undeclared determinants inside Obs2; D3 gives every determinant its own relation (Coordinator, Assignment, CompletionBand, BandFactor). Their key sets, reference graphs and update footprints (Task 4) are different, and renaming a table or splitting columns arbitrarily would not change them. All three were loaded with the **same** seeded logical state and queried with the same five questions, so the comparison in Tasks 8–10 isolates design.

TASK 2 – Identify the Functional Dependencies (2 Marks)

Derive functional dependencies from the explicit SwiftSpan Parcels rules, classify them as full/partial/transitive relative to {SenderID, ServiceID, DispatchWave}, and reject apparent dependencies caused by generated cohorts or one DispatchWave.

Answer:

2.1 Accepted dependencies and their business rules

#	Dependency	Business rule that makes it hold in every valid state	Type w.r.t. K = {SenderID, ServiceID, DispatchWave}
f1	SenderID → SenderName, RouteGroup	A sender has one name and one declared route group	Partial (SenderID ⊂ K)
f2	ServiceID → ServiceCode, ServiceName, VolumeWeight	A ServiceID identifies one catalog object	Partial (ServiceID ⊂ K)
f3	ServiceCode → ServiceID	ServiceCode is a declared unique business identifier (so ServiceID ↔ ServiceCode)	Partial / alternate-key link
f4	{SenderID, ServiceID} → CoordinatorID	A coordinator is assigned to a sender/service pair across all dispatch waves	Partial ({SenderID, ServiceID} ⊂ K)
f5	CoordinatorID → CoordinatorName, Hub	A coordinator has one name and one hub	Transitive via f4 (K → CoordinatorID → name, Hub)
f6	{SenderID, ServiceID, DispatchWave} → Completion	One observation has exactly one Completion value	Full (the observation key itself)
f7	Completion → ServiceBand	Fixed fictional value-to-band policy	Transitive via f6
f8	ServiceBand → RouteFactor	Fixed policy: each band has one factor	Transitive via f6, f7

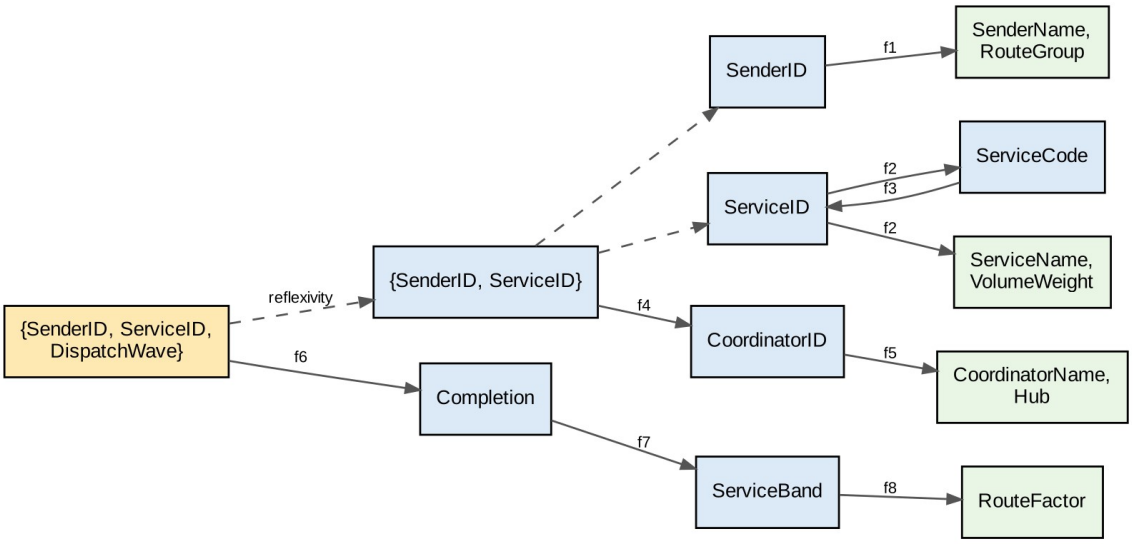


Figure 4 – Dependency graph (solid = declared; dashed = reflexivity); shaded boxes are the dependent facts

Full dependency: only f6, whose determinant is the whole key. **Partial** dependencies are f1–f4: their determinants are proper subsets of K, so Sender, catalog and assignment facts are repeated for every dispatch wave – this is what makes D1 violate 2NF. **Transitive** dependencies go through a non-key determinant: K → Completion → ServiceBand → RouteFactor, and K → {SenderID, ServiceID} → CoordinatorID → CoordinatorName/Hub; they are the reason D2 still repeats operator and policy facts.

2.2 Apparent dependencies that are rejected

I tested every ordered pair X → Y of single attributes on the generated 10,000-row Flat table (SQL `GROUP BY X HAVING COUNT(DISTINCT Y) > 1`). **24 pairs hold in the data**; those implied by the declared set are legitimate consequences (for example SenderID → SenderName). The following hold in this instance but are **not implied** by the declared rules, so they are **rejected**:

Apparent dependency in the data	Why it holds in the generated rows	Why it is rejected
SenderName → SenderID, RouteGroup	Generator names senders 'Sender 00001…' (all distinct)	Names are not declared unique; two senders may share a name
ServiceName → ServiceID, ServiceCode, VolumeWeight	Generator names services 'Service 01…' (all distinct)	ServiceName uniqueness is not a rule; ServiceCode (not name) is the declared alternate key
CoordinatorName → CoordinatorID, Hub	Generated coordinator names are distinct	Two coordinators may legitimately share a name
ServiceID / ServiceCode / ServiceName → RouteGroup	Each service belongs to exactly one generated cohort of five (G0–G7)	The case does not say a service is used by one group only; this is the generated RouteGroup cohort
RouteFactor → ServiceBand	Four bands have four different factors in the policy table	Policy could give two bands the same factor; only ServiceBand → RouteFactor is declared

One-wave illusion: inside DispatchWave = 4 alone, {SenderID, ServiceID} → Completion has **0 violations**, because each pair is observed once; across the four waves **2,500 of 2,500 pairs** have more than one Completion value. So Completion depends on the full key including DispatchWave, not on the pair. Likewise RouteGroup → ServiceID fails (8 of 8 groups have several services), so RouteGroup does not determine the catalog item – it only selects the required set in `Req`.

Dependencies are derived from rules, so each accepted one can be checked against a business rule; data patterns were used only to **reject** candidates, never to accept them.

TASK 3 – Derive Closures, Keys and a Minimal Cover Mathematically (6 Marks)

Prove the scope-to-operator-name and alternate-code-to-observation consequences with Armstrong's axioms, show the closures, find every candidate key, reduce the augmented cover, prove equivalence and classify prime/non-prime attributes.

Answer:

3.1 Armstrong proofs

(a) {SenderID, ServiceID} → CoordinatorName. (1) {SenderID, ServiceID} → CoordinatorID [given f4]. (2) CoordinatorID → CoordinatorName [f5, decomposition of CoordinatorID → CoordinatorName, Hub]. (3) {SenderID, ServiceID} → CoordinatorName [transitivity on 1, 2]. This is the 'redundant scope-to-name FD' of the augmented set – it is derivable, so it is not independent.

(b) {SenderID, ServiceCode, DispatchWave} → {SenderID, ServiceID, DispatchWave} → Completion. (1) ServiceCode → ServiceID [f3]. (2) {SenderID, ServiceCode, DispatchWave} → {SenderID, ServiceID, DispatchWave} [augmentation of 1 with SenderID, DispatchWave, plus reflexivity for the two attributes already on the left, and union]. (3) {SenderID, ServiceID, DispatchWave} → Completion [f6]. (4) {SenderID, ServiceCode, DispatchWave} → Completion [transitivity on 2, 3]. So the alternate code reaches the same observation and every dependent fact.

3.2 Closures (computed step by step)

Closure of X = {SenderID, ServiceID, DispatchWave}: start with X; apply **f1** adds RouteGroup, SenderName; **f2** adds ServiceCode, ServiceName, VolumeWeight; **f4** adds CoordinatorID; **f5** adds CoordinatorName, Hub; **f6** adds Completion; **f7** adds ServiceBand; **f8** adds RouteFactor. Result: all **14** attributes, so X is a superkey.

Closure of Y = {SenderID, ServiceCode, DispatchWave}: start with Y; apply **f1** adds RouteGroup, SenderName; **f3** adds ServiceID; **f4** adds CoordinatorID; **f5** adds CoordinatorName, Hub; **f6** adds Completion; **f7** adds ServiceBand; **f8** adds RouteFactor; **f2** adds ServiceName, VolumeWeight. Note f3 (ServiceCode → ServiceID) fires first and only then f2/f4/f6 can fire. Result: all **14** attributes, so Y is also a superkey.

3.3 Candidate keys and why proper subsets fail

A program enumerated all 2^{14} attribute subsets, kept the minimal ones whose closure is the whole relation, and found **exactly two candidate keys**:

- **K1 = {SenderID, ServiceID, DispatchWave}**
- **K2 = {SenderID, ServiceCode, DispatchWave}**

Proper subset	Closure size (of 14)	Why it fails
{SenderID, ServiceID}	10	DispatchWave missing – Completion, ServiceBand, RouteFactor are not reached
{SenderID, DispatchWave}	4	Only f1 applies (SenderName, RouteGroup)
{ServiceID, DispatchWave}	5	Only f2/f3 apply (ServiceCode, ServiceName, VolumeWeight)
{SenderID, ServiceCode} / {SenderID, DispatchWave} / {ServiceCode, DispatchWave}	10 / 4 / 5	Same pattern for K2

Every key must contain SenderID (nothing determines it), DispatchWave (nothing determines it) and one of ServiceID/ServiceCode (the pair is mutually determining) – so there are no other keys. **Prime attributes:** SenderID, ServiceID, ServiceCode, DispatchWave. **Non-prime attributes (10):** SenderName, RouteGroup, ServiceName, VolumeWeight, CoordinatorID, CoordinatorName, Hub, Completion, ServiceBand, RouteFactor.

3.4 Minimal cover of the augmented set

The supplied augmented set is $F^{+}_{a} = F \cup \{$ {SenderID, ServiceID} → CoordinatorName (redundant) , {SenderID, ServiceID, VolumeWeight} → CoordinatorID (VolumeWeight extraneous) $\}$. **Step 1 – singleton right sides:** split every FD so the right side is one attribute. **Step 2 – remove extraneous left attributes:** for {SenderID, ServiceID, VolumeWeight} → CoordinatorID, the closure of {SenderID, ServiceID} already contains CoordinatorID (f4), so **VolumeWeight is extraneous** and is deleted. **Step 3 – remove redundant**

FDs: {SenderID, ServiceID} → CoordinatorName is implied by f4 and f5 (proof 3.1(a)), so it is **removed**. The program's log confirms exactly these two reductions. Minimal cover (12 FDs):

```
SenderID -> SenderName          SenderID -> RouteGroup
ServiceID -> ServiceCode        ServiceCode -> ServiceID
ServiceID -> ServiceName        ServiceID -> VolumeWeight
{SenderID,ServiceID} -> CoordinatorID
CoordinatorID -> CoordinatorName    CoordinatorID -> Hub
{SenderID,ServiceID,DispatchWave} -> Completion
Completion -> ServiceBand       ServiceBand -> RouteFactor
```

Equivalence proof. (⇒) Each FD of the cover is either in F or a decomposition of one, so F implies the cover. (⇐) Computing closures under the cover reproduces each original FD (program check: cover ⊨ F is **True**, F ⊨ cover is **True**); the augmented set is equivalent to F too (**True**), because its two extra FDs are implied by F. Therefore F, the augmented set and the reduced cover imply one another.

Where the cover differs from the normalised schema: f2/f3 form a cycle (ServiceID ↔ ServiceCode); in synthesis they are merged into one relation with two candidate keys, not two relations.

TASK 4 – Analyse Update, Insertion and Deletion Anomalies (4 Marks)

Use real generated rows to demonstrate an orphan catalog fact, deletion of its final observation and inconsistent ServiceName copies in all three designs; measure rows affected by changing ServiceID=1 at the three sizes; compare allocated bytes and repeated facts, labelling estimates separately.

Answer:

4.1 Real rows used (generated, size 10,000)

SenderID	SenderName	RouteGroup	ServiceID	ServiceCode	ServiceName	Vol	Wave	CoordID	CoordinatorName	Hub	Comp	Band	RF
1	Sender 00001	G1	6	SVC-006	Service 06	4	1	14	Coordinator 0014	Hub-3	45	C	115
1	Sender 00001	G1	6	SVC-006	Service 06	4	2	14	Coordinator 0014	Hub-3	66	B	100
1	Sender 00001	G1	7	SVC-007	Service 07	2	1	15	Coordinator 0015	Hub-4	83	B	100
1	Sender 00001	G1	7	SVC-007	Service 07	2	2	15	Coordinator 0015	Hub-4	62	C	115

Sender 1 (group G1) has service SVC-006 observed in waves 1 and 2 with values 45 and 66: SenderName, RouteGroup, ServiceName, VolumeWeight, CoordinatorName and Hub are copied unchanged on both rows, while Completion legitimately changes. Band C/RouteFactor 115 is repeated for every observation with a value of 45–64.

4.2 Insertion anomaly – orphan catalog fact

Design	Insert catalog item 41 (SVC-041, 'Cold-chain Parcel') with no observation	Cause
D1 flat	**Rejected:** NOT NULL constraint failed: Flat.SenderID	ServiceID → ServiceName lives inside an observation row whose key columns (SenderID, DispatchWave) must exist
D2 partial	Accepted – stored in `Service` alone	Catalog facts have their own relation
D3 BCNF	Accepted – stored in `Service` alone	Same

4.3 Deletion anomaly – losing the last observation

Service 41 was given exactly one observation in each design, then that observation was deleted (rolled-back trial). **D1:** the query `SELECT ServiceID, ServiceName FROM Flat WHERE ServiceID=41` returned 1 row before the delete and **0 rows after it – the catalog fact ('Cold-chain Parcel') disappeared with the observation.** D2 and D3 still return `(41, Cold-chain Parcel)`. The responsible dependency is ServiceID → ServiceName, VolumeWeight (f2), stored inside a relation keyed by the observation key.

4.4 Update anomaly – inconsistent copies

D1: for sender 8 the ServiceName of ServiceID 1 was changed on one observation only; the result is two names for one service: ‘Service 01’ × 247, ‘Express Parcel’ × 1. Rows of that sender: wave 1 → Express

Parcel; wave 2 → Service 01; wave 3 → Service 01; wave 4 → Service 01. **D2:** ServiceName cannot diverge (one row in `Service`), but the operator name still can: after changing one Obs2 row, coordinator 1 appears as 'Coordinator 0001' × 171, 'Coordinator 1 (typo)' × 1; and RouteFactor of band D shows 130 × 1093, 115 × 1 – a policy value that contradicts f8. **D3:** one `Coordinator` row and one `BandFactor` row exist, so these inconsistencies cannot be represented. Responsible dependencies: f2 (D1), f5 and f7/f8 (D1 and D2).

4.5 Rows touched by changing ServiceID = 1 (measured, rolled-back trials)

Operation (rolled back)	Design	10,000	100,000	600,000
Change ServiceName of ServiceID=1	D1 flat	248	2,500	15,000
	D2 partial	1	1	1
	D3 BCNF	1	1	1
Renumber ServiceID 1 → 9001	D1 flat	248	2,500	15,000
	D3 BCNF (1 Service row; cascades)	1 (+62 Assignment, 248 Observation via cascade)	1 (+625 Assignment, 2,500 Observation via cascade)	1 (+3,750 Assignment, 15,000 Observation via cascade)
Rename CoordinatorName of ID 1	D1 / D2 / D3	172 / 172 / 1	160 / 160 / 1	160 / 160 / 1
Change RouteFactor of band D	D1 / D2 / D3	1,094 / 1,094 / 1	11,203 / 11,203 / 1	67,085 / 67,085 / 1

ServiceID = 1 appears in 248, 2,500 and 15,000 observations, i.e. exactly 1/40 of them, so D1's cost grows linearly with the population while D2/D3 stay at one row. In D3 the key renumbering is the one case where a single logical row still causes physical work: the `ON UPDATE CASCADE` rewrites the dependent Assignment, Observation and `Req` rows – that is the price of using a changeable natural-looking ID; stable identifiers avoid it. The operator update touches the same rows in D1 and D2 (the coordinator's own observations – 172, 160 and 160 here, because the generator creates one coordinator per ten actors), which shows that **D2 did not fix the operator anomaly**, and the policy update touches thousands of rows in D1/D2 but one row in D3.

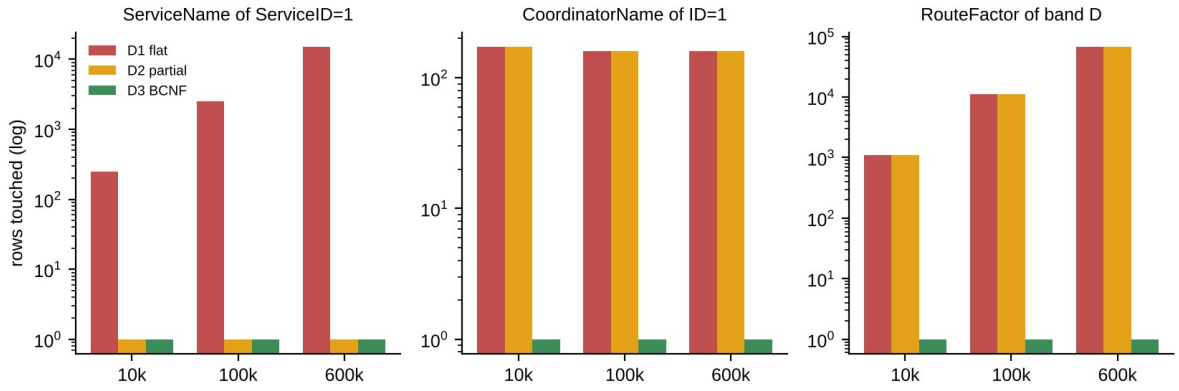


Figure 5 – Rows touched per update (log scale) – measured, rolled back

4.6 Repeated facts and allocated bytes – measurements and estimates kept apart

Fact	Distinct facts	Stored copies D1	Stored copies D2	Stored copies D3 (measured by COUNT)
ServiceName (10,000 / 600,000)	40	10,000 / 600,000	40 (in Service)	40
CoordinatorName	50 / 3,000	10,000 / 600,000	10,000 / 600,000	50 / 3,000
RouteFactor	4	10,000 / 600,000	10,000 / 600,000	4
SenderName	500 / 30,000	10,000 / 600,000	500 / 30,000	500 / 30,000

Size	D1 measured MB	D2 measured MB	D3 measured MB	D1 estimate MB	D2 estimate MB	D3 estimate MB
10,000	1.47	0.89	0.62	0.88	0.50	0.20
100,000	14.95	9.00	5.85	8.80	5.00	2.02
600,000	90.82	54.78	35.70	52.80	30.00	12.08

Measured = allocated SQLite pages × 4,096 B with all tested indexes; estimate = naive payload (integer 4 B, text length + 1).

Measured: D1 allocates 90.82 MB at 600,000 observations, D2 54.78 MB and D3 35.70 MB, i.e. D1 is 2.5× and D2 1.5× the BCNF design. **Estimated** (not measured): the naive payload model counts only column values; it is far below the measured pages because it ignores B-tree structure, page fill and the primary-key/secondary indexes (a SQLite table with a composite PK stores the key twice). The estimate is only used to show the ratio of repeated payload (D1 : D3 = 4.4×).

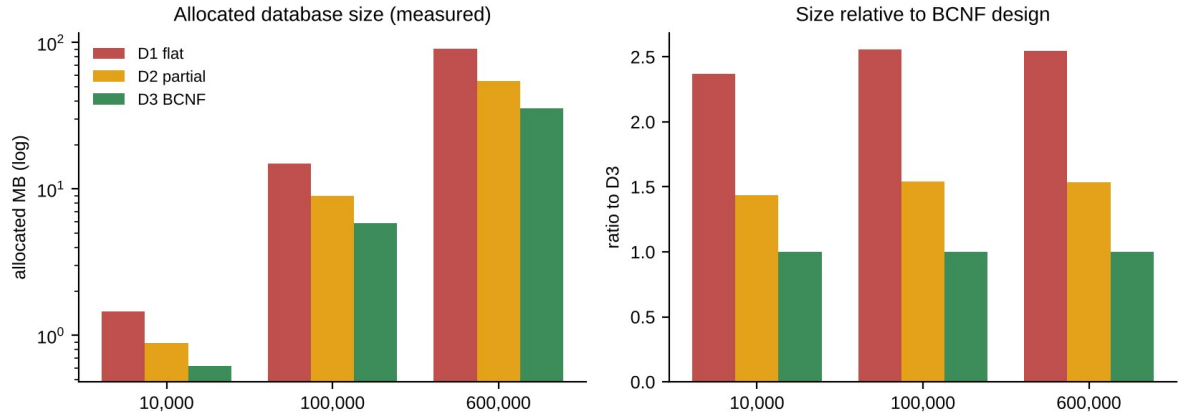


Figure 6 – Allocated database size of the three designs (measured)

TASK 5 – Determine Normal Forms up to BCNF (3 Marks)

Determine the normal form of every relation with actual determinants, show synthesis groups and binary decompositions, and examine an AB → D, D → B example that is 3NF without BCNF.

Answer:

5.1 Tests applied to every relation

Design	Relation	Candidate key(s)	2NF	3NF	BCNF	Violating determinant(s)
D1	Flat	K1, K2	✘	✘	✘	f1–f4 (partial), f5, f7, f8 (transitive)
D2	Sender	SenderID	✔	✔	✔	–
D2	Service	ServiceID, ServiceCode	✔	✔	✔	–
D2	Obs2	(SenderID, ServiceID, DispatchWave)	✘	✘	✘	{SenderID,ServiceID} → CoordinatorID (partial); CoordinatorID → Name, Hub; Completion → Band; Band → Factor
D3	Sender	SenderID	✔	✔	✔	–
D3	Service	ServiceCode, ServiceID	✔	✔	✔	–
D3	Coordinator	CoordinatorID	✔	✔	✔	–
D3	Assignment	(SenderID, ServiceID)	✔	✔	✔	–
D3	Observation	(DispatchWave, SenderID, ServiceID)	✔	✔	✔	–
D3	CompletionBand	Completion	✔	✔	✔	–
D3	BandFactor	ServiceBand	✔	✔	✔	–

Method. 1NF: all attributes are atomic (no repeating groups). 2NF: no non-prime attribute depends on a proper subset of a candidate key – tested with the projected FDs. 3NF: for every non-trivial X → A, X is a superkey or A is prime. BCNF: for every non-trivial X → A, X is a superkey. The results above come from the program (`fd_theory.py`), which projects F onto each relation. **D1** fails 2NF because f1–f4 are partial; **D2.Obs2** also fails 2NF (CoordinatorID is determined by part of the key) – its Sender and Service relations are in BCNF (Service has two keys, ServiceID and ServiceCode); **every D3 relation is in BCNF**, including `Assignment` (determinant = its key).

5.2 Dependency-preserving synthesis (3NF) and the binary BCNF decomposition

Synthesis groups (one relation per left side of the minimal cover, merging FDs with the same left side, then adding a key relation if none contains a candidate key): R1(SenderID, SenderName, RouteGroup) · R2(ServiceID, ServiceCode, ServiceName, VolumeWeight) [f2 and f3 merged: two keys] · R3(SenderID, ServiceID, CoordinatorID) · R4(CoordinatorID, CoordinatorName, Hub) · R5(SenderID, ServiceID, DispatchWave, Completion) [contains K1, so no extra key relation is required] · R6(Completion, ServiceBand) · R7(ServiceBand, RouteFactor). These seven relations are exactly D3.

Binary BCNF decomposition (each step splits on a violating determinant X → Y, producing R(X ∪ Y) and R(U − Y)): (1) Flat on ServiceBand → RouteFactor → BandFactor(ServiceBand, RouteFactor) and the rest; (2) on Completion → ServiceBand → CompletionBand(Completion, ServiceBand); (3) on CoordinatorID → CoordinatorName, Hub → Coordinator; (4) on SenderID → SenderName, RouteGroup → Sender; (5) on ServiceID → ServiceCode, ServiceName, VolumeWeight → Service; (6) on {SenderID, ServiceID} → CoordinatorID → Assignment, leaving Observation(SenderID, ServiceID, DispatchWave, Completion). The result is again D3, and all projected FDs are preserved because every left side is the key of the relation that holds its right side (program check: dependency preservation **True** for D1, D2 and D3).

5.3 Assignment-policy example: AB → D and D → B (3NF without BCNF)

Let R(A, B, D) with FDs **AB → D** and **D → B**. Candidate keys (computed): **{A,B}, {A,D}** – so B is prime. **3NF holds** (the only non-key determinant, D, determines the prime attribute B). **BCNF fails:** D → B has a non-superkey left side. BCNF decomposition on D → B gives **R1(D, B)** and **R2(A, D)**, which is lossless, but **AB → D is not preserved** (it can no longer be checked inside one relation; the projected FDs give only D → B) – so a join is needed to enforce the policy 'one D per (A, B)'. In SwiftSpan terms, think of A = operator, B = item, D = channel: an operator–item pair has one approved channel, and each channel belongs to one item. Trade-off: choose **3NF** if enforcing AB → D without joins matters; choose **BCNF** if the redundancy of repeating B beside D matters more. For the actual SwiftSpan dependencies no such conflict arises: BCNF and dependency preservation are both achieved (D3).

TASK 6 – Analyse Lossless Join, Dependency Preservation and Higher Normal Forms (3 Marks)

Run the chase for the chosen decomposition, contrast a wrong common-ServiceID split, test dependency enforcement, decompose independent multivalued facts to 4NF and construct a genuine three-projection join dependency for 5NF.

Answer:

6.1 Chase for D3 – a lossless witness

Tableau with one row per relation of D3 (rows R1…R7, a-symbols where the relation has the attribute, distinct b-symbols elsewhere). The chase applied the FDs in this order (program log): f1 equates RouteGroup; f1 equates SenderName; f2 equates ServiceCode; f2 equates VolumeWeight; f2 equates ServiceName; f4 equates CoordinatorID; f5 equates CoordinatorName; f5 equates Hub; f7 equates ServiceBand; f8 equates RouteFactor. After 10 equalisations **row 5 (Observation) became all a-symbols** – the chase has produced a tuple of distinguished symbols, so the decomposition is **lossless**. Intuitively: Observation → Assignment (f4) → Coordinator (f5), → Sender (f1) and Service (f2), and Completion → Band → Factor (f7, f8): every join is on a key of one side.

6.2 The wrong split on the common ServiceID

Wrong split: **R1(SenderID, SenderName, RouteGroup, ServiceID, ServiceCode, ServiceName, VolumeWeight)** and **R2(ServiceID, DispatchWave, CoordinatorID, CoordinatorName, Hub, Completion, ServiceBand, RouteFactor)**. Their intersection ServiceID is not a key of R1 or R2 (neither side's remaining attributes depend on it alone), so the chase can equate only the Service columns (f2) and **never** produces an all-a row (program: lossless = **False**). Real data, size 10,000 (SQLite): R1 has **2,500** distinct rows, R2 has **9,826** distinct rows, their natural join on ServiceID has **614,164 rows** while the original relation had **10,000** – so **604,164 spurious rows** appear; the join cannot recover which sender had which completion. The correct split joins on a key (SenderID, ServiceID, DispatchWave).

6.3 Dependency preservation – can the consequences still be enforced?

Original FD	D3 relation that enforces it	Enforceable without a join?
f1 SenderID → SenderName, RouteGroup	Sender (PK SenderID)	Yes
f2/f3 ServiceID ↔ ServiceCode, …	Service (PK ServiceID, UNIQUE ServiceCode)	Yes
f4 {SenderID, ServiceID} → CoordinatorID	Assignment (PK)	Yes
f5 CoordinatorID → Name, Hub	Coordinator (PK)	Yes
f6 key → Completion	Observation (PK)	Yes
f7 Completion → ServiceBand	CompletionBand (PK)	Yes
f8 ServiceBand → RouteFactor	BandFactor (PK)	Yes

Each FD becomes a PRIMARY KEY or UNIQUE constraint on one relation, so the database enforces all of them directly. Contrast: the example of Task 5.3 (AB → D) is the case where preservation is lost.

6.4 4NF – independent multivalued facts

Consider service 1 with **approved transport modes** {Air, Road} and **required document types** {Invoice, Manifest, Customs}, independent of each other: `ServiceTransport(ServiceID, Mode, Document)` must store the full cross product – **6 rows**:

ServiceID	Mode	Document
1	Air	Invoice
1	Air	Manifest
1	Air	Customs
1	Road	Invoice
1	Road	Manifest
1	Road	Customs

Dependencies: **ServiceID →→ Mode | Document** (an MVD, not an FD; the key is all three columns, so the relation is in BCNF but **not in 4NF**). Adding a mode requires adding three rows; deleting the last Invoice for Air leaves an inconsistent grid. **4NF decomposition:** ServiceMode(ServiceID, Mode) = {(1,Air), (1,Road)} (2 rows) and ServiceDocument(ServiceID, Document) = {(1,Invoice), (1,Manifest), (1,Customs)} (3 rows). The join returns the 6 original rows (program check: lossless = **True**), and each new relation is keyed by all its columns, so it is in 4NF.

6.5 5NF – a three-way approval rule (join dependency)

Rule: an **operator** may handle an **item** over a **channel** only if (operator, item), (item, channel) and (operator, channel) are each separately approved and the combination is approved. The relation Approval(Operator, Item, Channel) found by a seeded search has these 8 tuples (O = operators, I = items, C = channels):

Operator	Item	Channel
O1	I1	C2
O1	I2	C2
O1	I3	C2
O2	I1	C2
O2	I3	C1
O2	I3	C2
O3	I2	C2
O3	I3	C2

Projections: OI = 7 pairs, IC = 4 pairs, OC = 4 pairs. **The join of all three projections equals the original relation exactly** (program: **True**), so the relation satisfies the join dependency ⋈{OI, IC, OC}. **No pair of the projections is enough:** OI ⋈ IC alone has 10 rows – the extra 2 rows ('O1', 'I3', 'C1'), ('O3', 'I3', 'C1') are spurious until the third projection OC filters them out. The search also checked that **no MVD holds** (so the relation is already in 4NF and BCNF); the join dependency is not implied by any key, so the relation is **not in 5NF**. 5NF decomposition = the three binary relations OI, IC, OC (each is all-key and in 5NF). Practical

note: such cyclic rules are rare; if the business rule is really a single three-way approval (not decomposable), keep the ternary relation, which is then in 5NF.

TASK 7 – Formulate the Required Queries in Relational Algebra and Calculus(4 Marks)

Formulate the five operational questions in relational algebra (RA) and tuple relational calculus (TRC), at least two in domain relational calculus (DRC), marking aggregate extensions explicitly.

Answer:

Relations (D3): Sender(SenderID, SenderName, RouteGroup), Service(ServiceID, ServiceCode, ServiceName, VolumeWeight), Assignment(SenderID, ServiceID, CoordinatorID), Coordinator(CoordinatorID, CoordinatorName, Hub), Observation(SenderID, ServiceID, DispatchWave, Completion), CompletionBand(Completion, ServiceBand), BandFactor(ServiceBand, RouteFactor), Req(RouteGroup, ServiceID). **Notation:** σ select, π project, ⋈ natural join, − difference, ρ rename, **γ** grouping/aggregation (**an extension** – it is not part of classical RA). Let θ = 39 be the threshold. **Classical TRC/DRC cannot compute SUM, COUNT or AVG**, so queries 1 and 4 are marked (aggregate extension): the calculus part selects the tuples and the aggregate is applied to the selected set.

Q1 – actor 1, DispatchWave = 4, weighted adjusted index

RA. V = Observation ⋈ Service ⋈ CompletionBand ⋈ BandFactor. Rows = π_ServiceCode, Completion, RouteFactor, VolumeWeight (σ_SenderID=1 ∧ DispatchWave=4 (V)). (Aggregate extension) WAI = γ_ Σ(Completion·RouteFactor/100·VolumeWeight) / ΣVolumeWeight (Rows).

TRC. { t | ∃o ∈ Observation ∃v ∈ Service ∃cb ∈ CompletionBand ∃f ∈ BandFactor (o.SenderID = 1 ∧ o.DispatchWave = 4 ∧ v.ServiceID = o.ServiceID ∧ cb.Completion = o.Completion ∧ f.ServiceBand = cb.ServiceBand ∧ t.ServiceCode = v.ServiceCode ∧ t.Completion = o.Completion ∧ t.RouteFactor = f.RouteFactor ∧ t.VolumeWeight = v.VolumeWeight) } followed by the aggregate WAI over the result set.

DRC. { ⟨code, c, rf, vw⟩ | ∃svc, nm, b (Observation(1, svc, 4, c) ∧ Service(svc, code, nm, vw) ∧ CompletionBand(c, b) ∧ BandFactor(b, rf)) } (aggregate applied afterwards).

Executed result (identical in all three designs and at every size): SVC-006: completion 49, factor 115, weight 4; SVC-007: completion 76, factor 100, weight 2; SVC-008: completion 90, factor 90, weight 5; SVC-009: completion 48, factor 115, weight 3; SVC-010: completion 74, factor 100, weight 1. **WeightedAdjustedIndex = 68.1333.** (Σ Completion·RF/100·VW ÷ ΣVW over the five rows above.)

Q2 – senders meeting θ = 39 for every required service of their declared group

RA (division written with difference). Good = π_SenderID,ServiceID(Observation) − π_SenderID,ServiceID(σ_Completion<39(Observation)) (pairs observed, none below 39). Need = π_SenderID,ServiceID(Sender ⋈ Req). **Answer = π_SenderID(Sender) − π_SenderID(Need − Good).** A sender qualifies only if no required pair is unobserved or below 39; the observed-pair term prevents vacuous success for unobserved items.

TRC. { t | ∃s ∈ Sender (t.SenderID = s.SenderID ∧ ∀r ∈ Req (r.RouteGroup = s.RouteGroup → (∃o ∈ Observation (o.SenderID = s.SenderID ∧ o.ServiceID = r.ServiceID) ∧ ¬∃o2 ∈ Observation (o2.SenderID = s.SenderID ∧ o2.ServiceID = r.ServiceID ∧ o2.Completion < 39)))) }

DRC. { ⟨s⟩ | ∃n, g (Sender(s, n, g) ∧ ∀v (Req(g, v) → (∃w, c Observation(s, v, w, c) ∧ ¬∃w2, c2 (Observation(s, v, w2, c2) ∧ c2 < 39)))) }

Executed result: 153 senders at 10,000 observations, 1,594 at 100,000, 9,415 at 600,000 (about 30 % of senders at each size).

Q3 – observed services with no value below 39 in period 4

RA. P4 = σ_DispatchWave=4(Observation). **Answer = π_ServiceID(P4) − π_ServiceID(σ_Completion<39(P4)).** The first term restricts the answer to services actually observed in period 4, so unobserved catalog items cannot qualify vacuously.

TRC. { t | ∃o ∈ Observation (o.DispatchWave = 4 ∧ t.ServiceID = o.ServiceID ∧ ¬∃o2 ∈ Observation (o2.ServiceID = o.ServiceID ∧ o2.DispatchWave = 4 ∧ o2.Completion < 39)) }

DRC. { ⟨v⟩ | ∃s, c Observation(s, v, 4, c) ∧ ¬∃s2, c2 (Observation(s2, v, 4, c2) ∧ c2 < 39) }

Executed result: 14 services at 10,000 observations (3, 4, 7, 8, 12, 13, 16, 17), **none** at 100,000 and 600,000 – with 2,500 and 15,000 period-4 observations per size every service eventually has at least one value below 39, so an empty answer is the correct, non-vacuous result (the same in all three designs).

Q4 – coordinator means below the observation-weighted hub mean

RA (aggregate extension). J = Observation ⋈ Assignment ⋈ Coordinator. CM = γ_CoordinatorID, Hub; SUM(Completion)→s, COUNT(*)→n (J). HM = γ_Hub; SUM(s)/SUM(n)→hubmean (CM). **Answer = π_CoordinatorID, s/n, hubmean (σ_ s/n < hubmean (CM ⋈ HM)).** Weighting by n (observations) rather than averaging the coordinator means is what makes the hub mean 'observation-weighted'.

TRC (aggregate extension, over the derived relations CM and HM). { t | ∃c ∈ CM ∃h ∈ HM (c.Hub = h.Hub ∧ c.s / c.n < h.hubmean ∧ t.CoordinatorID = c.CoordinatorID) }. Classical calculus has no aggregate, so CM and HM are treated as given derived relations.

Executed result: 25 coordinators at 10,000 observations (of 50), 233 at 100,000 (of 500), 1,393 at 600,000 (of 3,000) – roughly half, as expected for a mean compared with its group mean.

Q5 – senders with sub-threshold observations in at least three distinct dispatch waves

RA (pure). B = π_SenderID, DispatchWave (σ_Completion<39(Observation)). Answer = π_SenderID (σ_ w1<w2 ∧ w2<w3 (ρ_w1(B) ⋈_SenderID ρ_w2(B) ⋈_SenderID ρ_w3(B))) with w1, w2, w3 the wave columns. (Extension form: σ_COUNT DISTINCT DispatchWave ≥ 3 (γ_SenderID; COUNT(DISTINCT DispatchWave) (σ_Completion<39(Observation))).)

TRC. { t | ∃o1, o2, o3 ∈ Observation (o1.SenderID = o2.SenderID ∧ o2.SenderID = o3.SenderID ∧ t.SenderID = o1.SenderID ∧ o1.Completion < 39 ∧ o2.Completion < 39 ∧ o3.Completion < 39 ∧ o1.DispatchWave < o2.DispatchWave ∧ o2.DispatchWave < o3.DispatchWave) }

Executed result: 33, 316 and 1,886 senders at the three sizes (about 6 % of senders).

TASK 8 – Implement the Database in SQL and Test It (4 Marks)

Implement and compare the three schemas with MySQL-oriented DDL and reproducible local verification: PK/FK/UNIQUE/NOT NULL/CHECK/restrictive deletes, the five queries in SQL, purpose-specific views with GRANT/REVOKE examples, identical data in all designs, and rolled-back anomaly updates.

Answer:

8.1 Reproducible testing procedure

- **Generate** one logical state with `random.Random(4211)` for 10,000 / 100,000 / 600,000 observations (500 / 5,000 / 30,000 actors × 5 required items × 4 waves).
- **Load** the same Python lists into D1, D2 and D3 (foreign keys off during the bulk load, then `PRAGMA foreign_key_check` → **0, 0, 0 violations** at the three sizes), then `ANALYZE`.
- **Verify identical state:** observation counts are equal in all designs (10,000 / 100,000 / 600,000), and the five queries return identical results in all designs (Task 9).
- **Execute** the anomaly updates inside `BEGIN … ROLLBACK`; the integrity tests inside rolled-back transactions; time each query five times and take the median.

8.2 Normalised schema D3 – executed DDL (SQLite dialect)

```
CREATE TABLE Sender (SenderID INTEGER PRIMARY KEY, SenderName TEXT NOT NULL, RouteGroup TEXT NOT NULL);
CREATE TABLE Service (ServiceID INTEGER PRIMARY KEY, ServiceCode TEXT NOT NULL UNIQUE, ServiceName TEXT NOT
NULL,
    VolumeWeight INTEGER NOT NULL CHECK (VolumeWeight BETWEEN 1 AND 5));
CREATE TABLE Coordinator (CoordinatorID INTEGER PRIMARY KEY, CoordinatorName TEXT NOT NULL, Hub TEXT NOT NULL);
CREATE TABLE BandFactor (ServiceBand TEXT PRIMARY KEY CHECK (ServiceBand IN ('A','B','C','D')), RouteFactor
INTEGER NOT NULL CHECK (RouteFactor > 0));
CREATE TABLE CompletionBand (Completion INTEGER PRIMARY KEY CHECK (Completion BETWEEN 0 AND 100),
    ServiceBand TEXT NOT NULL REFERENCES BandFactor(ServiceBand) ON UPDATE CASCADE ON DELETE RESTRICT);
CREATE TABLE Assignment (SenderID INTEGER NOT NULL REFERENCES Sender(SenderID) ON UPDATE CASCADE ON DELETE
RESTRICT,
    ServiceID INTEGER NOT NULL REFERENCES Service(ServiceID) ON UPDATE CASCADE ON DELETE RESTRICT,
    CoordinatorID INTEGER NOT NULL REFERENCES Coordinator(CoordinatorID) ON UPDATE CASCADE ON DELETE RESTRICT,
    PRIMARY KEY (SenderID, ServiceID));
CREATE TABLE Observation (SenderID INTEGER NOT NULL, ServiceID INTEGER NOT NULL, DispatchWave INTEGER NOT NULL
CHECK (DispatchWave BETWEEN 1 AND 4),
    Completion INTEGER NOT NULL REFERENCES CompletionBand(Completion) ON UPDATE CASCADE ON DELETE RESTRICT,
    PRIMARY KEY (SenderID, ServiceID, DispatchWave),
    FOREIGN KEY (SenderID, ServiceID) REFERENCES Assignment(SenderID, ServiceID) ON UPDATE CASCADE ON DELETE
RESTRICT);
CREATE TABLE Req (RouteGroup TEXT NOT NULL, ServiceID INTEGER NOT NULL REFERENCES Service(ServiceID) ON UPDATE
CASCADE ON DELETE RESTRICT, PRIMARY KEY (RouteGroup, ServiceID));
CREATE INDEX ix_obs_wave ON Observation(DispatchWave, ServiceID, Completion);
CREATE INDEX ix_asg_coord ON Assignment(CoordinatorID);
```

Constraints present: **PK** on every relation, **FK** with `ON DELETE RESTRICT` (nothing referenced can be deleted) and `ON UPDATE CASCADE` (key renumbering propagates), **UNIQUE** on `Service.ServiceCode`, **NOT NULL** on every column, **CHECK** constraints on Completion (0–100), DispatchWave (1–4), VolumeWeight (1–5), ServiceBand ∈ {A,B,C,D} and RouteFactor > 0. D1 and D2 use the same checks and keys (see Appendix B).

8.3 The five questions in SQL (D3 form; D1 and D2 forms differ only in the FROM clause)

Q1 – weighted adjusted index (window function)

```
SELECT v.ServiceCode AS ServiceCode, o.Completion AS Completion, bf.RouteFactor AS RouteFactor,
v.VolumeWeight AS VolumeWeight,
  ROUND(o.Completion*bf.RouteFactor/100.0, 2) AS AdjustedValue,
  ROUND(SUM(o.Completion*bf.RouteFactor/100.0*v.VolumeWeight) OVER () / SUM(v.VolumeWeight) OVER (), 4)
AS WeightedAdjustedIndex
FROM Observation o JOIN CompletionBand cb ON cb.Completion=o.Completion JOIN BandFactor bf ON
bf.ServiceBand=cb.ServiceBand JOIN Service v ON v.ServiceID=o.ServiceID WHERE o.SenderID=1 AND
o.DispatchWave=4 ORDER BY v.ServiceCode
```

Q2 – relational division by counting qualified required services

```
SELECT g.sid FROM (SELECT o.SenderID AS sid, s.RouteGroup AS rg, o.ServiceID AS svid, MIN(o.Completion)
AS m FROM Observation o JOIN Sender s ON s.SenderID=o.SenderID GROUP BY o.SenderID, s.RouteGroup,
o.ServiceID) g
WHERE g.m >= 39 AND g.svid IN (SELECT ServiceID FROM Req WHERE RouteGroup = g.rg)
GROUP BY g.sid, g.rg HAVING COUNT(*) = (SELECT COUNT(*) FROM Req r WHERE r.RouteGroup = g.rg) ORDER BY
g.sid
```

Q3 – grouped anti-condition (MIN ≥ 39) in period 4, non-vacuous because the group must exist

```
SELECT o.ServiceID AS ServiceID, MIN(o.Completion) AS MinCompletion, COUNT(*) AS Observations
FROM Observation o WHERE o.DispatchWave=4 GROUP BY o.ServiceID HAVING MIN(o.Completion) >= 39 ORDER BY
o.ServiceID
```

Q4 – coordinator mean vs observation-weighted hub mean

```
WITH cm AS (SELECT a.CoordinatorID AS cid, c.Hub AS hub, SUM(o.Completion) AS s, COUNT(*) AS n FROM
Observation o JOIN Assignment a ON a.SenderID=o.SenderID AND a.ServiceID=o.ServiceID JOIN Coordinator c
ON c.CoordinatorID=a.CoordinatorID GROUP BY a.CoordinatorID, c.Hub),
hm AS (SELECT hub, 1.0*SUM(s)/SUM(n) AS hubmean FROM cm GROUP BY hub)
SELECT cm.cid AS CoordinatorID, ROUND(1.0*cm.s/cm.n, 4) AS CoordinatorMean, ROUND(hm.hubmean, 4) AS
HubMean
FROM cm JOIN hm ON hm.hub = cm.hub WHERE 1.0*cm.s/cm.n < hm.hubmean ORDER BY cm.cid
```

Q5 – distinct waves with sub-threshold values

```
SELECT o.SenderID AS SenderID, COUNT(DISTINCT o.DispatchWave) AS BadWaves FROM Observation o
WHERE o.Completion < 39 GROUP BY o.SenderID HAVING COUNT(DISTINCT o.DispatchWave) >= 3 ORDER BY
o.SenderID
```

Query 2's anti-join form: `g.svid IN (SELECT ServiceID FROM Req …)` and a `HAVING COUNT() = (SELECT COUNT() FROM Req …)` that requires **all** required services to qualify. Because D1 holds all columns in one table, the same queries have 0 joins; D2 has up to 2 and D3 up to 6 (Task 9).

8.4 Purpose-specific views (executed) and GRANT/REVOKE (not executed)

```
CREATE VIEW v_actor_card AS
  SELECT o.SenderID, s.SenderName, v.ServiceCode, v.ServiceName, o.DispatchWave, o.Completion,
cb.ServiceBand
  FROM Observation o JOIN Sender s ON s.SenderID=o.SenderID JOIN Service v ON v.ServiceID=o.ServiceID
       JOIN CompletionBand cb ON cb.Completion=o.Completion;
CREATE VIEW v_ops_coordinator AS
  SELECT c.CoordinatorID, c.CoordinatorName, c.Hub, COUNT(*) AS Observations, ROUND(AVG(o.Completion),2)
AS MeanCompletion
  FROM Coordinator c JOIN Assignment a ON a.CoordinatorID=c.CoordinatorID
       JOIN Observation o ON o.SenderID=a.SenderID AND o.ServiceID=a.ServiceID
  GROUP BY c.CoordinatorID, c.CoordinatorName, c.Hub;
```

`v_actor_card` exposes 7 columns (SenderID, SenderName, ServiceCode, ServiceName, DispatchWave, Completion, ServiceBand) – no coordinator or hub data; `v_ops_coordinator` exposes aggregates only (CoordinatorID, CoordinatorName, Hub, Observations, MeanCompletion). Executed check for sender 1, wave 4: SVC-006 → 49 (band C); SVC-007 → 76 (band B); SVC-008 → 90 (band A); SVC-009 → 48 (band C); SVC-010 → 74 (band B). **A view does not enforce row identity:** `SELECT COUNT(*) FROM v_actor_card` returned 10,000 rows for 500 different senders – any role granted the view sees every sender's rows. Row-level restriction needs a `WHERE` bound to the account (for example a parameterised service layer or a view filtered by a session-to-sender mapping table); the view alone does not do it.

MySQL privilege examples (not executed – SQLite has no GRANT):

```
CREATE ROLE actor_reader, ops_analyst;
GRANT SELECT ON swiftspan.v_actor_card TO actor_reader;
GRANT SELECT ON swiftspan.v_ops_coordinator TO ops_analyst;
REVOKE ALL PRIVILEGES ON swiftspan.Observation FROM actor_reader;   -- base table stays
hidden
REVOKE UPDATE (CoordinatorName) ON swiftspan.Coordinator FROM ops_analyst;
```

8.5 Rolled-back anomaly trials

The title update (`UPDATE … ServiceName … WHERE ServiceID=1`) and operator-name update (`UPDATE … CoordinatorName … WHERE CoordinatorID=1`) were run for each design at each size inside `BEGIN … ROLLBACK`, and the affected-row counts were recorded (Task 4.5); afterwards the row counts were unchanged, so the same logical state could be reused by the next trial.

TASK 9 – Record Experimental Results (3 Marks)

Record rows touched, allocated pages and bytes with the tested indexes, median query times, expanded logical joins, actual integrity-test messages and result equivalence for all designs and sizes.

Answer:

9.1 Rows touched by catalog and operator changes

Change	Design	10,000	100,000	600,000
Catalog (ServiceName of ID 1)	D1 / D2 / D3	248 / 1 / 1	2,500 / 1 / 1	15,000 / 1 / 1
Operator (CoordinatorName of ID 1)	D1 / D2 / D3	172 / 172 / 1	160 / 160 / 1	160 / 160 / 1
Policy (RouteFactor of band D)	D1 / D2 / D3	1,094 / 1,094 / 1	11,203 / 11,203 / 1	67,085 / 67,085 / 1

9.2 Allocated pages and bytes (tested indexes included)

Observations	D1 flat	D2 partial	D3 BCNF	D1 ÷ D3 bytes	D2 ÷ D3 bytes
10,000	358 / 1.47 MB	217 / 0.89 MB	151 / 0.62 MB	2.37×	1.44×
100,000	3,651 / 14.95 MB	2,198 / 9.00 MB	1,428 / 5.85 MB	2.56×	1.54×
600,000	22,173 / 90.82 MB	13,373 / 54.78 MB	8,715 / 35.70 MB	2.54×	1.53×

Breakdown at 600,000 observations (dbstat, pages → MB): **D3** sqlite_autoindex_Observation_1 2,463 pages (10.09 MB); Observation 2,314 pages (9.48 MB); ix_obs_wave 2,210 pages (9.05 MB); Assignment 548 pages (2.24 MB); sqlite_autoindex_Assignment_1 536 pages (2.20 MB). **D1** Flat 12,263 pages (50.23 MB); sqlite_autoindex_Flat_2 3,463 pages (14.18 MB); sqlite_autoindex_Flat_1 2,463 pages (10.09 MB); ix_flat_wave 2,210 pages (9.05 MB). Indexes tested: PK/UNIQUE auto-indexes, `(DispatchWave, ServiceID, Completion)` for the period queries, and `CoordinatorID` for the operator grouping.

9.3 Median elapsed query time (ms, median of 5 runs) and expanded logical joins

Query	Observations	D1 flat ms	D2 partial ms	D3 BCNF ms
Q1	10,000	0.03	0.03	0.02
	100,000	0.02	0.02	0.02
	600,000	0.02	0.03	0.02
Q2	10,000	7.52	7.77	7.96
	100,000	86.76	87.95	87.13
	600,000	565.11	565.55	527.84
Q3	10,000	0.27	0.28	0.27
	100,000	2.48	2.40	2.37
	600,000	15.78	15.14	15.42
Q4	10,000	3.87	3.68	5.33
	100,000	48.12	45.88	63.05
	600,000	305.52	290.78	416.37
Q5	10,000	1.01	1.52	0.87
	100,000	12.25	10.65	9.29
	600,000	76.38	63.01	61.43

Query	Q1	Q2	Q3	Q4	Q5
Joins D1	0	0	0	1	0
Joins D2	1	1	0	1	0
Joins D3	3	1	0	3	0

'Expanded logical joins' = explicit `JOIN` operations needed to rebuild the attributes a query uses **before the optimizer** (counted on the SQL text of the executed statements): D1 never needs a join, D2 at most two (Sender, Service) and D3 up to three for the five queries and six for the full 14-column row (Task 12). `IN`/scalar sub-selects in Q2 are common to all three designs and are not counted.

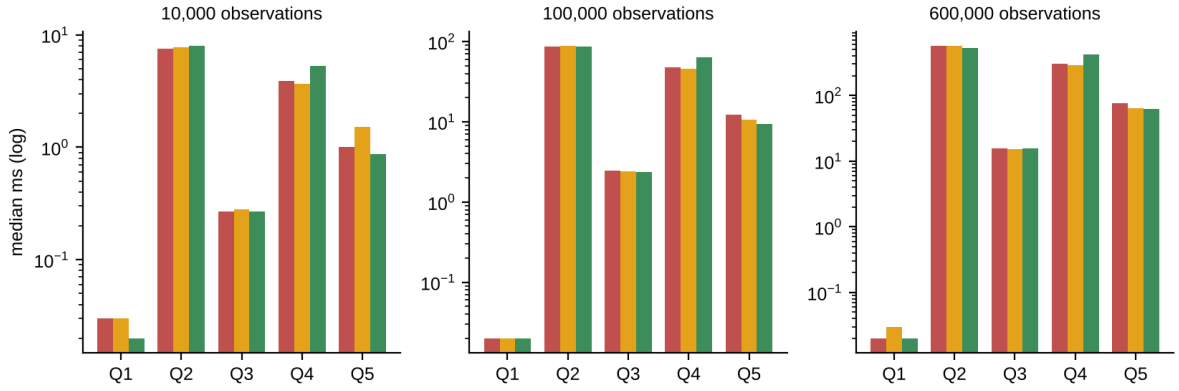


Figure 7 – Median query time per design (log scale, ms)

9.4 Actual integrity-test messages (D3, SQLite, executed in rolled-back transactions)

Test	Actual message
Duplicate observation key	UNIQUE constraint failed: Observation.SenderID, Observation.ServiceID, Observation.DispatchWave
Duplicate alternate key ServiceCode	UNIQUE constraint failed: Service.ServiceCode
Observation for an unassigned sender/service pair	FOREIGN KEY constraint failed
Completion outside 0..100	CHECK constraint failed: Completion BETWEEN 0 AND 100
Observation completion with no band row	FOREIGN KEY constraint failed
NULL ServiceName	NOT NULL constraint failed: Service.ServiceName
VolumeWeight outside 1..5	CHECK constraint failed: VolumeWeight BETWEEN 1 AND 5
DispatchWave outside 1..4	CHECK constraint failed: DispatchWave BETWEEN 1 AND 4
Assignment to unknown coordinator	FOREIGN KEY constraint failed
Band not in BandFactor	FOREIGN KEY constraint failed
Delete a sender that still has assignments	FOREIGN KEY constraint failed
Delete a coordinator that is still assigned	FOREIGN KEY constraint failed
Delete a service that is still observed	FOREIGN KEY constraint failed
Delete a band that is still referenced	FOREIGN KEY constraint failed

SQLite names the table/column for UNIQUE, NOT NULL and CHECK violations but returns only 'FOREIGN KEY constraint failed' for reference violations. In MySQL 8 the corresponding documented codes are 1062 (duplicate entry), 1048 (column cannot be null), 3819 (check constraint violated) and 1451/1452 (foreign-key failure) – **cited, not executed**.

9.5 Result equivalence across designs

Query	10,000 rows / hashes equal	100,000	600,000
Q1	5 rows; D1=D2=D3 **True**	5 rows; D1=D2=D3 **True**	5 rows; D1=D2=D3 **True**
Q2	153 rows; D1=D2=D3 **True**	1,594 rows; D1=D2=D3 **True**	9,415 rows; D1=D2=D3 **True**
Q3	14 rows; D1=D2=D3 **True**	0 rows; D1=D2=D3 **True**	0 rows; D1=D2=D3 **True**
Q4	25 rows; D1=D2=D3 **True**	233 rows; D1=D2=D3 **True**	1,393 rows; D1=D2=D3 **True**
Q5	33 rows; D1=D2=D3 **True**	316 rows; D1=D2=D3 **True**	1,886 rows; D1=D2=D3 **True**

For each query the result rows (rounded to 4 decimals, sorted) were serialised and hashed; the hashes of D1, D2 and D3 are **identical in all 15 query/size combinations**, which supports that the three schemas hold the same logical state and that the decompositions are lossless in practice. (This is evidence for the tested data, not a proof – the proof is the chase in Task 6.)

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Explain how the measurements agree with or qualify the dependency and normal-form analysis; relate repeated updates to determinants, assess the join trade-off using the actual timings, and explain index, cache and in-memory effects without inventing production guarantees.

Answer:

10.1 Repeated updates and retained redundancy

Theory predicted that an update to a fact stored under a **non-key determinant** touches one row per copy. The measurements agree: the ServiceName change touched 15,000 rows in D1 at 600,000 observations (= observations of ServiceID 1, i.e. N/40, the copies of f2 in a relation keyed by the observation) and exactly 1 row in D2 and D3 where f2 has its own key. The operator-name update touched 160 rows in **D2 as well as D1** – the experiment confirms the theory that D2 is still only 1NF: CoordinatorID is a determinant of CoordinatorName but is not a key of Obs2. The RouteFactor update touched 67,085 rows in D1 and D2 and **one** in D3, matching the transitive chain Completion → ServiceBand → RouteFactor. The anomalies were not merely possible: the rolled-back trials produced two names for one service (D1) and two factors for one band (D2).

One result **qualifies** the theory: the operator update touches about 160–170 rows at all three sizes (172, 160, 160) instead of growing with N, because the generator creates one coordinator per ten actors, so each coordinator handles about 200 observations at every size. Redundancy is proportional to the **copies of the**

fact that is changed, not to the table size – a property of this instance, whereas the policy and catalog updates scale with N.

10.2 Join cost vs redundancy – actual timings

At 600,000 observations Q4, which needs three joins in D3 (Assignment, Coordinator and the hub join) against one in D1 and D2, took 416 ms against 306 ms in D1 (1.36× slower) and 291 ms in D2 – the join price is real but moderate. Q5 was clearly **faster** in D3 and Q2 within run-to-run noise of D1 (see below): (528 vs 565 ms; 61 vs 76 ms), This is because their needed columns come from the narrow Observation table (four integer columns) so far fewer pages are scanned: the 14-column D1 rows cost 2.5× the bytes. Q1 is a primary-key range look-up and Q3 a covering-index scan, so they cost almost nothing in every design (≤ 16 ms). Normalisation therefore does not mean 'slower': wide rows make scans slower, joins make multi-fact queries slower, and the winner depends on the query's column footprint.

10.3 Index, cache and in-memory effects (no production guarantees)

Every figure comes from a single-process SQLite engine on a shared 2-core container whose file cache is warm after the load and `ANALYZE`; the five runs of one query differ by roughly 3–25 % (shared container; the median is reported and every run is stored in `results.json`), so differences below that band are noise. Several effects therefore flatter the numbers: (a) the primary-key and `(DispatchWave, ServiceID, Completion)` indexes turn Q1/Q3 into B-tree look-ups – the same would be needed in MySQL/InnoDB, where the clustered primary key also matters; (b) the 91 MB D1 file fits in memory, so no disk reads occur – a cold cache or a disk-based server would widen the gap between the wide D1 scans and the narrow D3 scans; (c) SQLite executes in-process with no network, lock manager, connection pool or concurrent writers, so the measured microseconds must **not** be read as server latency or throughput. The experiments establish **relative** behaviour of the designs on identical data; they cannot predict absolute MySQL performance.

TASK 11 – Recommend and Defend the Final Schema (4 Marks)

Recommend a relational schema based on correctness, update cost, measured behaviour and role scope; defend normal forms, losslessness and preservation; cite this version's results; separate implemented constraints and views from unexecuted MySQL privileges; state the costs.

Answer:

11.1 Recommendation

Choose D3 (the seven-relation BCNF design) plus the small reference table Req(RouteGroup, ServiceID). It is the only design in which every declared dependency f1–f8 is enforced by a primary or unique key, so none of the update anomalies of Task 4 can be represented; it is chosen on the dependencies and the measured behaviour, not on the normal-form label alone (a BCNF design that was ten times slower on the stated queries would have needed a different answer).

11.2 Defence

- **Normal forms:** all D3 relations are in BCNF (Task 5, program-verified); D1 is 1NF, D2's Obs2 is 1NF.
- **Lossless:** the chase for D3 yields an all-distinguished row (Task 6.1); the five queries return identical results in all designs at all sizes (Task 9.5).
- **Dependency preservation:** all eight FDs map to a key of one relation (Task 6.3), so the database enforces them without joins.
- **Update cost (measured):** one row for each of the catalog, operator and policy updates, against 15,000, 160 and 67,085 rows in D1 at 600,000 observations.
- **Storage (measured):** 35.70 MB vs 90.82 MB (D1) and 54.78 MB (D2) at 600,000 observations – D3 is 2.5× smaller than D1.
- **Query cost (measured):** D3 is within 1.36× of the fastest design on the worst query (Q4) faster on Q5 and within noise on Q2.

11.3 What was executed and what was not

Executed and observed: keys, foreign keys with restrictive deletes, UNIQUE/CHECK/NOT NULL (Task 9.4), the five queries and the two views, and the observation that a view shows all rows to anyone with SELECT on it. **Not executed:** MySQL DDL, `CREATE ROLE`, `GRANT`/`REVOKE`, MySQL error numbers and MySQL timings. The role scope in the recommendation (actors see `v_actor_card`, operations see

`v_ops_coordinator`) is a **design** that the privilege statements would implement – the privilege behaviour itself is untested here.

11.4 Costs of the recommendation

- **Policy changes** are cheap in the data (one row) but still need business control: moving a boundary (for example band C starting at 45) means updating CompletionBand rows (about 20 rows per boundary shift) and re-checking dependents; stored derived values would go stale.
- **Derived fields:** ServiceBand and RouteFactor are no longer stored on observations; they are derived by two joins. Storing them again would reintroduce f7/f8 redundancy (Task 4.4) – only allowed in a controlled read projection (Task 12).
- **Additional joins:** reconstructing the full 14-attribute row takes 6 joins; the measured price is ~36 % on the heaviest multi-fact query and about 7.2 µs per single-sender look-up (Task 12).
- **Constraint overhead:** foreign-key checks and cascaded renumbering add write cost; with `ON UPDATE CASCADE`, renumbering ServiceID 1 rewrote its dependent Assignment and Observation rows (Task 4.5), so keys should be treated as immutable.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed requirement: a promotion generates 55,000 sender tracking look-ups per hour, with reconciled handover corrections committed every six hours. Reassess the design and defend RETAIN, MODIFY or REDESIGN.

Answer:

A. Impact analysis (2 marks)

Changed workload. Before, reads and writes were assumed to be balanced operational queries and updates. Now the dominant operation is a **read-mostly single-sender tracking look-up** (one SenderID → that sender's five services with name, coordinator, hub, latest completion, band and factor), and writes arrive as a **batch of reconciled corrections committed every six hours** (4 commits a day) instead of continuous small updates. The corrections touch the same facts studied in Task 4: coordinator names, completions of the latest wave and, occasionally, policy factors.

Requests per second. 55,000 look-ups per hour ÷ 3,600 s = **15.28 requests per second** on average (about one request every 65 ms). If the promotion is bursty, a planning assumption of a 3× peak gives ≈ 46 requests/s – **that factor is my assumption, not a given**. The local look-up time measured below (≈ 34 µs) is not the server's throughput: a real service adds network, connection handling, concurrent lock and log activity; I do not divide 1 s by the local time to claim 'N requests per second'.

When a controlled read projection is acceptable despite repeated facts. When (i) the source data stay normalised and authoritative, (ii) the projection is rebuilt or refreshed only from the source, never edited directly, (iii) it is refreshed in the same transaction as the correction (or atomically after it) so readers never see half an amendment, and (iv) a validation query proves it equals the recomputed join. Then its repeated facts are derived, disposable data rather than a second source of truth.

B. Evaluation of the original design (2 marks)

Look-up of one sender's latest cards	Expanded joins	10,000 (µs)	100,000 (µs)	600,000 (µs)
D3 normal (six joins + MAX subquery)	6	26.2	43.4	33.8
D2 partial (two joins)	2	23.8	25.2	27.1
D1 flat (permanent duplicate)	0	20.5	23.8	26.6
D3 + indexed read card (ReadCard)	0	17.8	20.5	33.2

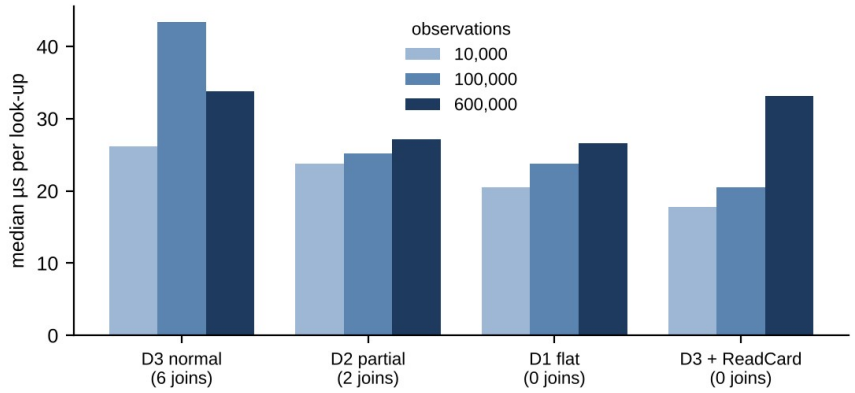


Figure 8 – Median local look-up time of four read paths (2,000 random senders × 5 repetitions)

What the numbers show. The indexed six-join look-up is already fast: 33.8 µs at 600,000 observations against 33.2 µs for the card – the card is only 2 % faster, because every join in the normal path is a primary-key probe; the card removes joins (6 → 0) but not the index probe itself. The four paths returned identical rows for 300 random senders (0 mismatches).

What a local benchmark can and cannot establish. It shows that, for this data, the query plan is index-driven and that the join count is not what limits a single look-up. It cannot establish capacity for 15.3–46 requests/s on a shared MySQL server (network, concurrency, buffer-pool misses, replication and lock contention are absent), nor that look-ups never block while corrections are committed.

Option	What it is	Stale data?	Redundancy / integrity	Measured effect
Ordinary view	Stored query over D3	Never (computed on read)	None; no extra storage	Same cost as the D3 normal look-up (6 joins on every read)
Transactional materialized read card	Table `ReadCard` filled from D3 in the amendment transaction	Only if refresh is skipped; detected by validation	Repeated facts, but derived and rebuildable; read-only for users	+13.95 MB at 600,000 (+39 % of D3); look-up 33 µs
Permanently duplicated source table (D1)	Flat table is the only copy	Never stale, but inconsistent copies possible	Full redundancy (Task 4): corrections touch many rows	Look-up 27 µs; the 6-hour amendment alone touches 960 + 67,085 rows

C. Adaptation – decision: RETAIN, with a measured and removable read card (2 marks)

Decision: retain the D3 BCNF schema and the normal read path. The new workload changes how often and how the data are accessed, not what is true of them: f1–f8 are business rules (a sender has one name, a coordinator one hub, a band one factor), so a higher look-up rate or a batch correction schedule cannot invalidate them, and Tasks 3–6 remain valid unchanged. The measured evidence also gives no reason to leave the schema: the normal six-join look-up costs 34 µs locally and a six-hourly correction touches **one row per corrected fact** (coordinator name, band factor) plus the corrected observations – 1,500 corrected observations, 5 renamed coordinators and a policy change were applied to the 600,000-observation source in **18 ms**. I did not choose the flat table, because the same batch would rewrite 960 + 67,085 repeated cells.

Contingency (physical structure only, deploy only if a MySQL load test fails the latency target): a derived, read-only table `ReadCard` that holds the latest wave of each sender/service pair. It adds no dependency (it is a join of source relations) and is rebuilt from the source:

```
CREATE TABLE ReadCard (            -- derived snapshot, never edited by users
  SenderID INT NOT NULL, ServiceID INT NOT NULL,
  ServiceCode VARCHAR(12) NOT NULL, ServiceName VARCHAR(60) NOT NULL,
  SenderName VARCHAR(60) NOT NULL, RouteGroup VARCHAR(4) NOT NULL,
  CoordinatorName VARCHAR(60) NOT NULL, Hub VARCHAR(12) NOT NULL,
  DispatchWave TINYINT NOT NULL, Completion TINYINT NOT NULL,
  ServiceBand CHAR(1) NOT NULL, RouteFactor INT NOT NULL,
  PRIMARY KEY (SenderID, ServiceID));  -- the look-up index
```

- **Snapshot consistency (executed demonstration, SQLite WAL, 100,000 size):** while a writer transaction had renamed a coordinator and rebuilt the card but not committed, a second connection read **25,000 rows and the old name 'Coordinator 0014'**; after the rollback the card was unchanged. The correction batch and any card refresh must therefore be **one transaction**. In MySQL/InnoDB the equivalent is `REPEATABLE READ` with the refresh in the same transaction (documented behaviour, not executed).

- **Amendment refresh:** every six hours apply the corrections, then rebuild the card (`DELETE` + `INSERT … SELECT`) or refresh only changed rows. At 600,000 observations the batch made **18,227 card rows stale**; a full rebuild took **1.23 s**, the incremental refresh **1.05 s**; both left 0 stale rows.
- **Validation:** after each refresh run `recompute EXCEPT card` and `card EXCEPT recompute`; both returned **0** rows at all sizes (600,000: 3.5 s). A non-zero result blocks publication of the snapshot.
- **Measured storage:** card 0.24 / 2.29 / 13.95 MB at the three sizes (2,500 / 25,000 / 150,000 rows); total database 49.65 MB vs 35.70 MB without (+39 %).

D. Evidence-based defence (2 marks)

Proven properties: D3 is in BCNF, lossless and dependency-preserving (Tasks 5–6), so every correction in the six-hourly batch is checked by keys and foreign keys. **Observed local behaviour:** normal look-up 34 µs against 33 µs for the card at 600,000 observations (a 2 % gain; at 10,000 observations the gain was 32 %), a full card rebuild 1.2 s, the source batch 18 ms. **Planning input:** 15.3 requests/s on average (≈46/s at an assumed 3× burst – my assumption) and four correction batches a day.

Why retain rather than modify or redesign. A modification must earn its extra storage (+39 %), refresh job and validation step; the measurements show that it removes joins (6 → 0) but not the key probe that dominates the look-up, so at 600,000 observations it saves almost nothing locally. Redesigning to the flat table would read about as fast (27 µs) but turn each correction into thousands of rewritten copies that can diverge (Task 4). **What this evidence cannot show:** that a shared MySQL server meets the latency target at 15–46 requests/s with concurrent look-ups and a commit; the local benchmark has no network, lock contention or cold cache. That is why the card is specified and measured as a removable projection: if a MySQL load test fails, add it without touching the logical design. Assumptions: 3× burst factor, one latest wave per card row, correction batch and refresh in one transaction, SQLite substituting for MySQL.

Mark distribution

Task	Marks	Task	Marks
1 – Alternative schemas	5	7 – Relational algebra and calculus	4
2 – Functional dependencies	2	8 – SQL implementation and testing	4
3 – Closures, keys, minimal cover	6	9 – Experimental results	3
4 – Anomaly analysis	4	10 – Theory vs experiment	4
5 – Normal forms	3	11 – Final schema recommendation	4
6 – Lossless join, 4NF, 5NF	3	Subtotal (Tasks 1–11)	42
12 – Re-evaluation design challenge	8	TOTAL	50

Signature of the student:



APPENDIX A – Runner extracts (run_swiftspan.py and fd_theory.py)

`run_swiftspan.py` (SQLite, seed 4211) generates the data, builds the three designs, runs all trials and writes `results.json`; `fd_theory.py` computes closures, keys, the minimal cover, normal forms, the chase and the 4NF/5NF examples. The data generator and the query builder are reproduced; both complete files accompany this report.

```
def generate(n_obs):
    rng = random.Random(SEED)
    actors = n_obs // (5 * WAVES)
    n_coord = max(40, actors // 10)
    services = [(s, f"SVC-{s:03d}", f"Service {s:02d}", (s * 3) % 5 + 1) for s in range(1, N_SERVICES + 1)]
    req = [(f"G{g}", g * 5 + k) for g in range(N_GROUPS) for k in range(1, 6)]
    reqmap = {}
    for g, s in req:
        reqmap.setdefault(g, []).append(s)
    senders = [(i, f"Sender {i:05d}", f"G{i % N_GROUPS}") for i in range(1, actors + 1)]
    coords = [(c, f"Coordinator {c:04d}", f"Hub-{c % 6 + 1}") for c in range(1, n_coord + 1)]
    assign, obs, flat = [], [], []
    svc = {s[0]: s for s in services}
    for sid, sname, grp in senders:
        for svid in reqmap[grp]:
            cid = 1 + ((sid * 7 + svid) % n_coord)
            assign.append((sid, svid, cid))
            cn, hub = coords[cid - 1][1], coords[cid - 1][2]
            mean = 50 + (svid * 7) % 31
            for w in range(1, WAVES + 1):
                comp = max(0, min(100, int(round(rng.gauss(mean, 14)))))
                b, rf = band_of(comp)
                obs.append((sid, svid, w, comp))
                s = svc[svid]
                flat.append((sid, sname, grp, svid, s[1], s[2], s[3], w, cid, cn, hub, comp, b, rf))
    return dict(senders=senders, services=services, coords=coords, assign=assign, obs=obs, flat=flat, req=req,
                bands=[(c, band_of(c)[0]) for c in range(101)], factors=[(b, rf) for _, _, b, rf in BANDS])

def Q1(d):
    q = QB(d); sid, wave, comp, rf, vw, code = (q.c(x) for x in ("SenderID", "DispatchWave", "Completion",
"RouteFactor", "VolumeWeight", "ServiceCode"))
    return f"""SELECT {code} AS ServiceCode, {comp} AS Completion, {rf} AS RouteFactor, {vw} AS VolumeWeight,
  ROUND({comp}*{rf}/100.0, 2) AS AdjustedValue,
  ROUND(SUM({comp}*{rf}/100.0*{vw}) OVER () / SUM({vw}) OVER (), 4) AS WeightedAdjustedIndex
FROM {q.frm()} WHERE {sid}=1 AND {wave}=4 ORDER BY {code}"""



def Q2(d):
    q = QB(d); sid, rg, sv, comp = (q.c(x) for x in ("SenderID", "RouteGroup", "ServiceID", "Completion"))
    return f"""SELECT g.sid FROM (SELECT {sid} AS sid, {rg} AS rg, {sv} AS svid, MIN({comp}) AS m FROM {q.frm()} GROUP
BY {sid}, {rg}, {sv}) g
WHERE g.m >= {THRESH} AND g.svid IN (SELECT ServiceID FROM Req WHERE RouteGroup = g.rg)
GROUP BY g.sid, g.rg HAVING COUNT(*) = (SELECT COUNT(*) FROM Req r WHERE r.RouteGroup = g.rg) ORDER BY g.sid"""



def Q3(d):
    q = QB(d); sv, wave, comp = (q.c(x) for x in ("ServiceID", "DispatchWave", "Completion"))
    return f"""SELECT {sv} AS ServiceID, MIN({comp}) AS MinCompletion, COUNT(*) AS Observations
FROM {q.frm()} WHERE {wave}=4 GROUP BY {sv} HAVING MIN({comp}) >= {THRESH} ORDER BY {sv}"""



def Q4(d):
    q = QB(d); cid, hub, comp = (q.c(x) for x in ("CoordinatorID", "Hub", "Completion"))
    return f"""WITH cm AS (SELECT {cid} AS cid, {hub} AS hub, SUM({comp}) AS s, COUNT(*) AS n FROM {q.frm()} GROUP BY
{cid}, {hub}),
hm AS (SELECT hub, 1.0*SUM(s)/SUM(n) AS hubmean FROM cm GROUP BY hub)
SELECT cm.cid AS CoordinatorID, ROUND(1.0*cm.s/cm.n, 4) AS CoordinatorMean, ROUND(hm.hubmean, 4) AS HubMean
FROM cm JOIN hm ON hm.hub = cm.hub WHERE 1.0*cm.s/cm.n < hm.hubmean ORDER BY cm.cid"""



def Q5(d):
    q = QB(d); sid, wave, comp = (q.c(x) for x in ("SenderID", "DispatchWave", "Completion"))
    return f"""SELECT {sid} AS SenderID, COUNT(DISTINCT {wave}) AS BadWaves FROM {q.frm()}
WHERE {comp} < {THRESH} GROUP BY {sid} HAVING COUNT(DISTINCT {wave}) >= 3 ORDER BY {sid}"""



QUERIES = {"Q1": Q1, "Q2": Q2, "Q3": Q3, "Q4": Q4, "Q5": Q5}
```

APPENDIX B – MySQL 8.0 DDL for the three designs (submission form, not executed)

```
CREATE TABLE Sender (SenderID INT PRIMARY KEY, SenderName VARCHAR(60) NOT NULL, RouteGroup VARCHAR(4) NOT NULL);
CREATE TABLE Service (ServiceID INT PRIMARY KEY, ServiceCode VARCHAR(12) NOT NULL UNIQUE, ServiceName VARCHAR(60) NOT
NULL,
  VolumeWeight TINYINT NOT NULL, CONSTRAINT ck_vw CHECK (VolumeWeight BETWEEN 1 AND 5));
CREATE TABLE Coordinator (CoordinatorID INT PRIMARY KEY, CoordinatorName VARCHAR(60) NOT NULL, Hub VARCHAR(12) NOT
NULL);
CREATE TABLE BandFactor (ServiceBand CHAR(1) PRIMARY KEY, RouteFactor INT NOT NULL,
  CONSTRAINT ck_band CHECK (ServiceBand IN ('A','B','C','D')), CONSTRAINT ck_rf CHECK (RouteFactor > 0));
CREATE TABLE CompletionBand (Completion TINYINT PRIMARY KEY, ServiceBand CHAR(1) NOT NULL,
  CONSTRAINT ck_comp CHECK (Completion BETWEEN 0 AND 100),
  FOREIGN KEY (ServiceBand) REFERENCES BandFactor(ServiceBand) ON UPDATE CASCADE ON DELETE RESTRICT);
CREATE TABLE Assignment (SenderID INT NOT NULL, ServiceID INT NOT NULL, CoordinatorID INT NOT NULL,
  PRIMARY KEY (SenderID, ServiceID),
  FOREIGN KEY (SenderID) REFERENCES Sender(SenderID) ON UPDATE CASCADE ON DELETE RESTRICT,
  FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID) ON UPDATE CASCADE ON DELETE RESTRICT,
  FOREIGN KEY (CoordinatorID) REFERENCES Coordinator(CoordinatorID) ON UPDATE CASCADE ON DELETE RESTRICT);
CREATE TABLE Observation (SenderID INT NOT NULL, ServiceID INT NOT NULL, DispatchWave TINYINT NOT NULL, Completion
TINYINT NOT NULL,
  PRIMARY KEY (SenderID, ServiceID, DispatchWave), CONSTRAINT ck_wave CHECK (DispatchWave BETWEEN 1 AND 4),
  FOREIGN KEY (SenderID, ServiceID) REFERENCES Assignment(SenderID, ServiceID) ON UPDATE CASCADE ON DELETE RESTRICT,
  FOREIGN KEY (Completion) REFERENCES CompletionBand(Completion) ON UPDATE CASCADE ON DELETE RESTRICT);
CREATE TABLE Req (RouteGroup VARCHAR(4) NOT NULL, ServiceID INT NOT NULL, PRIMARY KEY (RouteGroup, ServiceID),
  FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID) ON UPDATE CASCADE ON DELETE RESTRICT);
CREATE INDEX ix_obs_wave ON Observation(DispatchWave, ServiceID, Completion);
CREATE INDEX ix_asg_coord ON Assignment(CoordinatorID);
-- D1: Flat(SenderID, SenderName, RouteGroup, ServiceID, ServiceCode, ServiceName, VolumeWeight, DispatchWave,
CoordinatorID,
--          CoordinatorName, Hub, Completion, ServiceBand, RouteFactor), PRIMARY KEY (SenderID, ServiceID,
DispatchWave),
--          UNIQUE (SenderID, ServiceCode, DispatchWave)
-- D2: Sender, Service (above) + Obs2(SenderID, ServiceID, DispatchWave, CoordinatorID, CoordinatorName, Hub,
Completion,
--          ServiceBand, RouteFactor), PRIMARY KEY (SenderID, ServiceID, DispatchWave), FKs to Sender and Service
```